

MySmartFridge Application

Introduction to Python - Final Project

Alexandre Galle

Yang Qin

Elvira Wild Martin

1. Introduction

With the pace of modern life accelerating, many people often face difficulties in planning their daily meals. They purchase new ingredients but are unsure how to combine them with the items already in their fridge, which can lead to food waste or disorganized cooking plans. To address this issue, we developed MySmartFridge, an application that can scan supermarket receipts using OCR technology, extract the purchased ingredients, and integrate them with the existing items in the user's fridge, providing a reliable data foundation for subsequent recipe generation.

Building on this, MySmartFridge can suggest recipes based on the ingredients available in the virtual fridge, helping users plan their meals more efficiently and make better use of their food. To improve the accuracy of processing complex receipt information, the application combines rule-based text parsing with semantic post-processing using a local large language model (Ollama). Through this process, the system automatically organizes the inventory of ingredients. On top of that, it provides personalized and practical recipe recommendations, thereby enhancing the convenience and efficiency of home cooking.

2. Informal specifications

2.1 Main functionalities

The final application implements the following core functionalities:

1. Receipt Scanning & Item Extraction
 - Users upload a JPG/PNG image of a grocery receipt
 - The Tabscanner OCR API extracts item names and quantities
 - Text is normalized (lowercase, whitespace trimmed) and LLM refinement pipeline is used to remove non-food items and further standardize food items
 - Items are stored in user's virtual fridge
2. Per-User Inventory Management
 - Each user has an isolated fridge stored as JSON (no database required)
 - Since JSON is stored locally, there is no need for password protection
 - Users can manually add/remove items and quantities
 - Users can clear their entire fridge inventory
 - Users can view their inventory
3. Recipe Suggestion Engine

- Based on available ingredients in the fridge, Ollama (llama3.2 model) generates 3 different recipe suggestions
 - Each recipe includes name, ingredient list and cooking steps
4. Recipe Cooking & Tracking
- Users select a recipe to cook, which automatically deducts used ingredients from the fridge
 - Full recipe details (name, ingredients, steps, timestamp) are logged to recipe history
 - Users can view all previously cooked recipes
5. Activity Logging
- All user actions (scan, cook, clear) are logged with timestamps
 - Users can view their activity history per session

2.2 Constraints and Scope Boundaries

- Only web application available, not accessible through mobile/smartphone app
- Receipt images must be pre-saved on the user's computer before upload
- Ollama server must be running and accessible via command-line
- OCR accuracy depends on receipt image quality and legibility
- Recipe generation quality depends on Ollama model capabilities and available ingredients

2.3 Assumptions

- Receipts provided are reasonably clear and not heavily damaged or blurred.
- Items are in English or will be recognized or normalized by OCR or AI refinement pipeline
- Platform for local/personal use only (no authentication required, simple session-based username)
- Basic pantry staples (salt, oil, pepper, water, spices) are assumed to be available
- Users have Ollama and llama3.2 model pre-installed locally

3. Development Plan

3.1 Initial Milestones

When we started the project, we set the following **milestones**:

- An algorithm able to transform a receipt into a list of ingredients with their associated quantity
- A way to create a 'virtual fridge' in Python able to track the actual items in the fridges of different users

We then searched which **challenges** we would encounter when coding these two features and we found the following:

- Differentiate between two users' fridge

- Manage the items in the fridges based on the recipes they will choose
- Implement an OCR scanning feature that would read receipts, parse the results and automatically store the items of the receipt in the virtual fridge
- Store two equivalent items that have different names within same category in fridge inventory

3.2 Revised Milestones

Then, after discussing with the professor and thinking about new/different features, we set the actual milestones of the project. Based on feasibility and feedback our revised milestones evolved:

Milestone 1	Virtual Fridge Class	Ability to manage different users with methods to add, remove, clear, deduct items used by recipe undertaken and load items from user input (and finally from receipt upload)
Milestone 2	OCR Scanning	Implement an OCR scanning feature that would read receipts, parse the results and automatically store the items of the receipt in the virtual fridge
Milestone 3	Recipe Generation	Integrate local LLM that uses available items in fridge as input and outputs 3 possible recipes that can be cooked
Milestone 4 (optional)	User-Interface	Web-based or if possible, mobile graphical user-interface that allows for simple uploading of receipt pictures, viewing fridge and suggested recipes and history. Optionally, also a dashboard that uses cooking history to display data on foods eaten or similar

3.3 Stages

After development, our final versioning was the following:

V0: Minimal Working System

The V0 contains the virtual fridge with features such as adding, removing, and clearing items and the OCR scanning feature, transforming a receipt into a properly formatted dictionary of items. The functions in main.py that can be run interactively through a terminal integrate fridge and scanning, since receipt items are automatically added to the user's fridge. Thus, the first two milestones were accomplished in V0.

V1: Recipe Suggestion System

The V1 was a big step in the project since it added a main feature (the recipe suggestion feature), accomplishing the third milestone, and transforming a minimal working system into a real application that has useful functions. The class Fridge also received an upgrade, with

a new method 'deduct_by_recipe' able to determine which items were used by a recipe, and thus deleting these items from the fridge. However, we knew that it still had limitations due to the lack of UI, and the difficulty of generating recipes. The user had to know how to interact with the terminal and the results were not appealing and stable.

V2: Web Application

The V2 is a properly functioning web app with clear defined features (suggesting recipe and scanning receipts). It enables any user to interact with the application without interacting with the terminal (except for launching the app). Other secondary features were added, such as clearing the entire fridge, manually adding and removing items, and directly uploading receipt pictures from the computer, instead of inserting a path.

3.4 Rationale for the order of implementation

The first component implemented was the Fridge class, which represents the core data structure of the project. This class is a pure application of OOP principles we covered in class, even though some external libraries were used. All subsequent functionalities interact directly or indirectly with this class, making it the central element of the system.

Once the core object-oriented structure was in place, we implemented the feature that initiated the project: transforming a receipt image into a structured dictionary using OCR. This step required exploring external APIs and handling noisy, unstructured data, which went beyond the scope of the course material and introduced new technical challenges.

Finally, we added two higher-level features aimed at improving usability and user engagement. The first was a recipe suggestion system based on the contents of the virtual fridge, allowing users to make practical use of the extracted data. The second was the development of a user interface using Flask, enabling end users to interact with the system in a simple and intuitive way.

After basic modules were finished, they were imported into a central main.py module, where functions were written that combined classes, as well as functions for the app functionality, such as prompting username, prompting menu. This central module was gradually adapted as new features were developed, until it was swapped out by the app.py module that contained a restructured logic to conform with flask application.

4. Design & Implementation

4.1 Inventory Management

Fridge.py was one of the first modules that was implemented in v0 of our project, to allow the creation of the virtual fridge and the management of inventory. The fridge class started off with each instance including the attributes, username, inventory (dictionary) and number of ingredients (which was later removed as it was redundant and only helpful for manual testing).

The first basic methods implemented were `__repr__`, `__str__` and `__contains__`. Then, methods like `add_item()`, `remove_item()`, `clear_fridge()` and `has_item()` were implemented to

allow for basic manipulation of the fridge's inventory. For simplicity and normalization, `lower()` and `strip()` are applied to each item before being added to or removed from inventory and when checking if contained in the fridge. Following this, we had the basis for writing other methods, like `load_from_receipt(receipt_dic)`, to be used in combination with the `api.py` module.

After manual testing of the functionalities, when building the `main.py` application, when running across the problem of keeping users inventory throughout different functions, we implemented a JSON storage solution that uses the `save_fridge()` and `load_fridge(user)` methods. For the latter a class method was used, since the inventory linked to a username is loaded from the local JSON file `'all_fridges.json'` and a new `Fridge()` object is created with this username and inventory. If this username does not exist (`KeyError`), we create a new object with it. Initially, in v0 and v1, we used two JSON files, one for storing all usernames and one for fridge inventory, until we realized this was redundant and directly used the inventory one for usernames, as these are the dictionary keys. When collaborating we also realized that the code needed to include the generation of a new JSON file if none is available (`FileNotFoundError`). Therefore, we always use `if os.path.exists(file), else open(file, 'w'): json.dump()` or sometimes, `try:/except`.

After the recipe suggerer was implemented in v1, we added `deduct_by_recipe(recipe)`. In order to assure compatibility, the prompt was adjusted to output an ingredient the number of times that it is needed. Thus, if two tomatoes are needed, a dictionary with recipe ingredients mentions tomato twice. According to this, the `deduct_by_recipe()` method deducts ingredients one-by-one from fridge after recipe has been selected.

4.2 Scanning and Parsing

The scanning and parsing architecture was designed iteratively as the handling of the TabScanner API was unknown at the beginning of the project. Firstly, we created the `TabScannerClient` with as a single argument the `API_KEY` (loaded from a `dotenv` file for privacy on GitHub) to define all the methods we will use for the scanning. We developed this class with only one public method available to foster simplicity between the team members: the `'scan()'` method with the image path as the only required argument. This method outputs a dictionary with the following structure: *{item 1: qty (in kg or in units), ... , item n = qty}*.

The `scan()` method relies on three main internal functions that form the receipt processing pipeline.

Processing: `process_receipt()`

This method is the only component that directly interacts with the TabScanner API. It sends the receipt image to the API for OCR processing and periodically polls the result endpoint until the processing is completed. Since the API offers 200 calls per month in the free versions, we only put 2 attempts to retrieve with a 20s interval (not very convenient, but to test it we had to use the API a lot).

This method returns a raw and noisy dictionary and includes basic error handling to manage failed uploads, missing tokens, or timeouts.

Extraction: `extract_items()`

This method takes as input the output of `process_receipt()` and parses it.

The initial OCR result contains many non-essential elements such as the supermarket name, the date and time of purchase, or other metadata. The first role of this function is therefore to filter out these irrelevant lines and retain only item descriptions and their associated quantities.

However, the extracted items are often noisy and inconsistent. For example, the API may return entries such as: 'tomatoes offres vrac 750g': 1, 'Linguines vrac 500g': 1..., 'Chicken breast': 1, 'chicken breast': 2

To address this issue we create another function called `clean_ocr_item()` outside the class to use it within `extract_items`. This function performs several normalization steps:

- lowercasing and trimming item names
- removal of promotional keywords (for example: *offer*, *maxi*)
- removal of packaging information and formatting noise
- detection of weight-based quantities (kg or g)

For weighted items, quantities are converted into kilograms (e.g. "bananas 740g: 1" becomes "bananas: 0.74"), which avoids inconsistencies in downstream inventory management. Regular expressions (re library) were extensively used to implement these transformations.

Lastly, for items wrongfully identified as ingredients (VAT, Thank you for your purchase...), we created a boolean condition with `is_candidate_food_item`. This method only allows an item to be added to the dictionary if it does not contain forbidden keywords (we identified them ourselves e.g., 'Merci', 'VAT', 'Thank you'...). Thereby, this function solves the problem.

In conclusion, this method outputs a cleaner version of the dictionary returned by TabScanner.

Refinement Pipeline: `ai_refine_list()`

The last function in this class is the `ai_refine_list` which uses the local LLM Ollama to uniformize and remove undesirable items that could not be removed with deterministic methods. We wanted to avoid the usage of an LLM but figured out that certain operations could not be performed with standard python libraries. As a result, we parsed as much as possible using deterministic methods and we only gave to Ollama tasks that were not efficient if done without an LLM.

For instance, TabScanner considers kraft bag: 1, chewing gum Freedent:2 as items and thus includes it in the dictionary (outputted by `extract_items`). To handle these cases, the LLM is used strictly as a post-processing step. It receives the cleaned dictionary produced by the previous functions and is guided by a carefully designed prompt with strict constraints. The model is instructed to normalize and translate item names, remove undesirable items, and

return a dictionary with unchanged quantities, thereby limiting the risk of hallucinations. See lines 165 to 204 of `api.py` for detailed prompt.

If the post-processing fails, it returns the dictionary input itself, verifying that the items are accepted by the boolean method `is_candidate_food_item`.

In practice, creating an instance of the client and calling:

```
a = TabScannerClient()  
a.scan(image_path)
```

triggers the complete pipeline.

The image is sent to TabScanner and undergoes OCR to be transformed into a dictionary item: quantity. Then, the dictionary is parsed and lastly, is sent to Ollama for post-processing.

Scanning Datastructure

We chose to use a dictionary as the core data structure, as it was the best way to represent the relationship between an item and its quantity. Each item name is used as a key, while the corresponding quantity is stored as a value. This structure allows constant-time average access, insertion, and updates, which is particularly useful for aggregating items extracted from OCR outputs.

Regarding the algorithms, two main approaches are used in the project:

- **Rule-based text normalization algorithm** based on regular expressions is applied to OCR-extracted receipt lines. This deterministic approach was chosen after manually analyzing a large number of receipts and identifying recurrent OCR errors and noise patterns (such as prices, units, promotions, and formatting artifacts discussed earlier). A rule-based solution allows these issues to be addressed explicitly and produces controllable outputs that we designed ourselves, which is essential for reliability and debugging.
- **LLM-based semantic refinement step** is applied to handle limitations that could not be reliably solved using deterministic rules alone. During receipt analysis, we identified semantic issues such as item translation, product category normalization, and the removal of non-food items, which require contextual understanding. For this reason, a local large language model (Ollama) is used only as a post-processing step, refining the output of the rule-based pipeline without modifying quantities. This hybrid approach combines the robustness of deterministic algorithms with the flexibility of machine learning, while maintaining control over the overall system behavior.

4.3 Recipe Generation

The recipe generation functionality is fully implemented within a single module, `recipe_suggester_ollama.py`. This module does not depend on any other custom modules in the project. It relies only on Python standard libraries (`json`, `re`) and the third-party library `requests`, which is used to communicate with the local Ollama service. The design objective of this module is intentionally narrow and well defined: to take a fridge inventory dictionary as

input, invoke a local large language model, and return structured recipe data that can be consumed directly by the application.

Module Structure

The module contains only one core class and one internal helper function:
RecipeSuggesterOllama (core business class); `_extract_json_array()` (private helper method).

Core Class: RecipeSuggesterOllama

This class encapsulates the complete recipe generation workflow and serves as the single entry point for all recipe requests within the system.

The initializer is responsible for a single task: storing the name of the Ollama model to be used (default: llama3.2). The model name is intentionally exposed as a configurable parameter rather than being hard-coded. This design allows different models to be tested or swapped without modifying the core business logic.

The `suggest(self, inventory, n_recipes=3)` method has the following features:

1. Inventory Data Preprocessing

The inventory parameter is received as a dictionary with the structure: `{ingredient_name: quantity}`. In the current implementation, only the dictionary keys (ingredient names) are used. All ingredient names are concatenated into a comma-separated string for subsequent prompt construction. This design ensures that the recipe generation logic focuses solely on ingredient availability rather than precise quantities, which aligns well with the practical requirements of LLM-based generation.

2. Prompt Construction (Strictly Structured)

The prompt is constructed directly in the code as a multi-line string and exhibits several explicit constraints: the model role is clearly defined (professional chef API); the list of available ingredients is explicitly provided; the number of recipes to generate is specified; strict output rules are imposed: the output must be valid JSON; the result must be a list; only English is allowed; no explanatory or descriptive text is permitted; the expected JSON field structure is explicitly illustrated. The purpose of this prompt is not to enhance creativity, but to tightly constrain the model's output format so that it can be reliably parsed by the program.

3. Local Ollama API Invocation

Recipe generation is carried out through a single HTTP POST request to the local Ollama API using `requests.post()`. The request is configured with `stream=False` to ensure that a complete response is returned in one operation, and the `"format": "json"` option is enabled to encourage the model to produce structured output. In addition, a timeout of 45 seconds is explicitly specified to prevent the request from blocking indefinitely in case of unexpected delays or failures. This step represents the only interaction between the module and an external service.

4. Model Output Retrieval and Debug Logging

- the response field is extracted from the JSON returned by Ollama;
- the raw model output is printed to the console for debugging and inspection;
- debug logging is intended for development and does not affect the main program flow.

5. JSON Cleaning and Parsing

Since LLM outputs do not always strictly adhere to formatting constraints, a dedicated cleaning step is implemented:

- the internal `_extract_json_array()` method is invoked;
- a JSON array or object is extracted from the raw text output;
- `json.loads()` is used to convert the cleaned text into Python data structures.

This stage is critical for ensuring system robustness and stability.

6. Structural Validation and Normalized Return

After parsing, the resulting data structure is validated to ensure consistency. If the parsed output is a non-empty list, it is returned directly. If the output consists of a single dictionary, it is normalized by wrapping it into a list. In cases where the structure does not conform to the expected format, an exception is raised and handled by the centralized error-handling mechanism. This guarantees that the `suggest()` method always returns a list, simplifying downstream application logic.

7. Exception Handling and Fail-Safe Design:

All exceptions (network, parsing, format) are caught internally, returning a structured “error recipe” to avoid crashes.

Internal Helper Method: `_extract_json_array(self, text)`

- Removes Markdown code blocks using regex.
- Extracts the first complete JSON array (`[...]`); if none, falls back to a single JSON object (`{...}`).
- Improves robustness against noisy or imperfect model outputs.

Data Structures

In terms of data structures, we chose a standardized JSON format (list of dictionaries) as the core structure for both input and output.

- Input: Accepts a dictionary in the same format as the Fridge class `{ingredient: quantity}`, ensuring seamless integration between modules.
- Output: The model is required to return a list of recipe objects: `[{"name": "...", "ingredients": [...], "steps": [...]}]`. This structure was chosen because it can be directly parsed and rendered as cards in the front-end (Webapp) without any additional conversion steps.

At the algorithm level, we mainly adopt a hybrid parsing strategy:

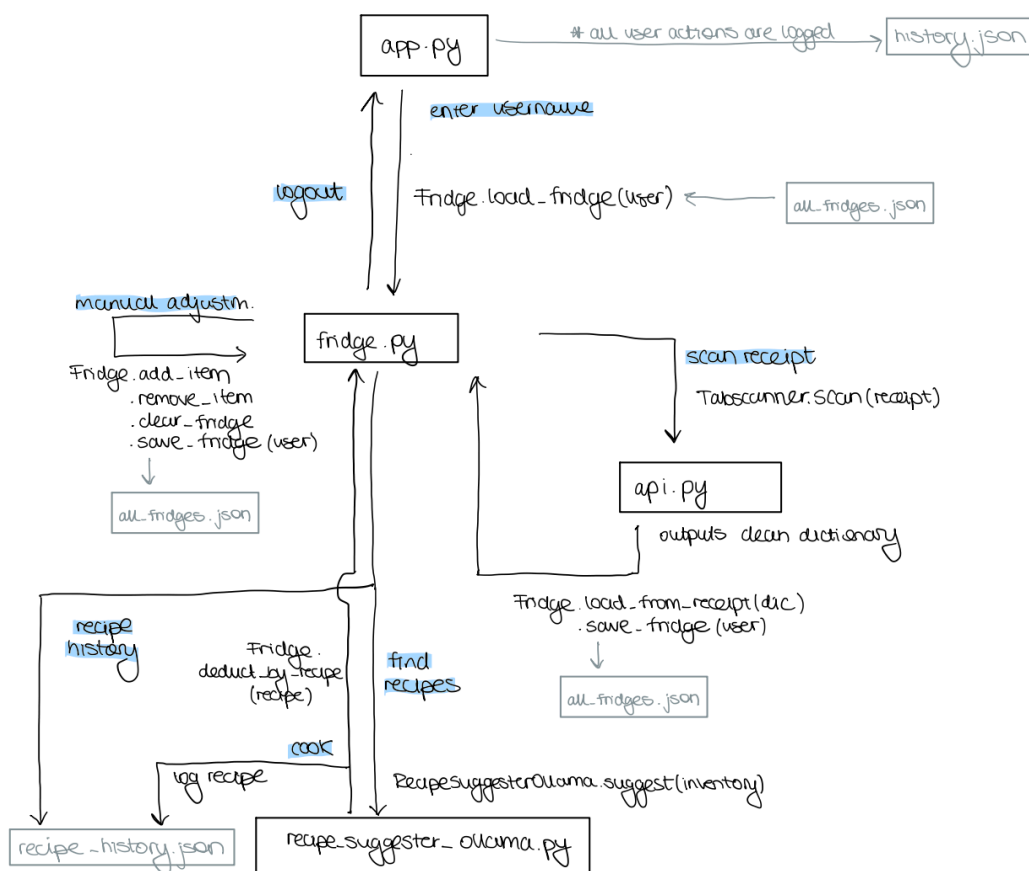
- **Regex-based extraction:** This is the most critical part of the code. Although the prompt instructs the model to output JSON, Ollama (Llama 3.2) often includes Markdown markers such as ````json` or introductory text. To handle this, we use regular expressions (`re.search` and `re.sub`) in `_extract_json_array()`. This prevents

json.loads() from crashing. The regex algorithm accurately extracts the content between the first [and the last], discarding any irrelevant noise.

- **Defensive programming and fault-tolerant design:** We wrap the generation process in a try-except block to handle network timeouts or parsing errors. If generation fails, the system does not crash but returns a predefined “error recipe” (“Chef is Sleeping”). Since the local LLM is resource-intensive and responds slowly (we set a 45-second timeout in the code), this fault-tolerant mechanism is crucial for maintaining a smooth user experience.

4.4 Relationship between modules

The following diagram visually shows the relations between different modules and the use of classes:



4.5 Notes on the Use of AI

During the implementation, we used LLM, not only as a feature of our application but also to help with coding. V0 was mostly written by us, and AI was used to occasionally enhance code or solve momentaneous challenges or doubts. In V1, it was used to write testing files and adapting the code to the use of the local ollama. We globally used it in a supervised manner for the insertion of logging.

Lastly, we relied on it for navigating the creation of the GUI interface using the flask package, as suggested in the slides. Here, we firstly created a very basic interface with an index and pages for each feature and then used AI to enhance the html scripts, for nicer graphics, and

then to implement logging directly in the app, and optimizing it using static methods to not unnecessarily create instances for this class.

4.6 Testing

For testing, in v0 we used manual testing. Where we either ran our classes using main functions inside the script or using ipython and interactively testing all features.

In v1, we also used interactive testing but added some automatized testing that can be found in the tests folder.

For v2, we used the tests from v1, which covered some functionalities of the fridge, api and recipe suggestion classes. However, some of the v1 tests have been rewritten to test new features of the classes.

For the web application (Flask), we created a basis with an LLM. We then performed tests directly in the file app.py, tweaking ourselves some parameters and observing the changes directly in the web application.

5. User guide

5.1 Previous installations

Setting up Virtual Environment

1. Open a python IDE, navigate to the v2 folder and create a virtual environment:
`python -m venv .venv`
2. Activate the virtual environment:
`.\.venv\Scripts\Activate.ps1`
3. Upgrade pip (package manager) to latest version and install project dependencies:
`python -m pip install -U pip`
`pip install -r requirements.txt`

Getting a Tabscanner API Key

In order to use the Tabscanner OCR API that allows us to scan the supermarket receipts and convert them into a dictionary, we need to create an account to get our API Key.

1. Go to <https://tabscanner.com/> and click login to create new or login into existing account.
2. Make sure *server status* displays 'Online'.
3. Go to *API Keys*, click to view default key and copy key.
4. Go to .env file in v2 folder and past key into `API_KEY` parameter

The free version of this API allows for 200 calls per month, which should suffice for non-commercial use of the application.

Ollama LLM

For recipe suggestion, the web application makes use of the local ollama LLM. Make sure there is at least 1.5 GB storage available on local disk before installing.

1. Open new terminal and navigate to v2 folder
2. To install, run `winget install Ollama.Ollama`
3. Add ollama to system path if not automatically done
4. Pull the model with `ollama pull llama3.2`
5. Run ollama with `ollama serve`

5.2 Running GUI of Web-Application

1. Navigate to v2 folder
2. Run `python app.py`

Expected Output:

```
Python
* Serving Flask app 'app'
WARNING: This is a development server. Do not use it in a production
deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5001
* Running on http://192.168.1.199:5001
Press CTRL+C to quit
```

3. To access the application, open browser and navigate to highlighted address in terminal output above. If this does not work try: <http://localhost:5001>

Navigation Flow

1. Login Page: Enter username → Click login
2. Dashboard: Three main action cards
 - Update Stock: Upload receipt image → Click "Scan Receipt"
 - Smart Chef: Click "Find Recipes" → View suggested recipes
 - Kitchen Ops: Click "View Logs" or "Empty Fridge"
3. Manual Adjustments: Click "Manual adjustments" link → Add/remove items
4. Recipe History: Click "Recipes cooked" link → View cooking history

Input for Testing

In the project folder, in the example_receipts folder, there are two receipt images that can be uploaded for scanning, in order to test functionality.

5.3 Error Handling

To enter debugging mode in v2, set the `DEBUGGER` parameter in `app.py` to `True`.

Some common errors and solutions are:

Tabscanner API key error: ensure ``.env`` has ``API_KEY`` and the file is in ``final_v2/``.

Empty fridge message: add items manually or upload a receipt first.

Port already in use: set a different port in ``app.py`` when calling ``app.run``.

Ollama Error

None

 Ollama Not Running

Ollama is not running. Please start Ollama with
'ollama serve' in a terminal and ensure it's added
to your PATH.

Quick fix:
ollama serve

[← Return to Dashboard](#)

To fix:

- Follow the guidelines of the error
- May be because ollama was not run in the same directory as application
- Check ollama is properly running in terminal
- If nothing works, refresh page and try again

6. Evaluation & discussion

6.1 Challenges

The main difficulty we encountered was first to use a GitHub repository with different branches to work remotely at the same time (it was our first real-scale project). We learned (sometimes the hard way) how to pull, commit, push our changes in distinct branches, enabling us to retrieve the changes from team members. Consequently, the three of us could focus on a specific task.

Pythonically speaking, the parsing of the items dictionary was tricky. We designed the pipeline for a long time because we wanted to avoid the usage of an LLM. Even with the use of this LLM, finding all possible issues by reading receipts for hours was a real challenge. We addressed it mostly with the regular expressions library that was very efficient and adapted to this issue.

A second important challenge was the database management: keeping each user's fridge inventory across sessions and combining this with the fridge class and its methods. To address it, as mentioned in section 4.1, we used a json file to store users' inventory after

modifications, and created a class method that loads from the json file and creates a new instance corresponding to the user with loaded inventory.

Lastly, integrating Ollama was definitely one if not the hardest challenge. The first version of Ollama we chose systematically made our computer crash. We solved this by switching to 3.2 after extensive attempts. Finding the right prompt for Ollama both in the recipe suggestions and in the parsing was long. We addressed this issue by giving it TASKS and RULES, and by creating exceptions to be raised to handle wrong outputs (not a strict JSON dictionary for instance).

6.2 Possible extensions or improvements

As mentioned in the milestones, we had the idea to create a dashboard using the recipe history, that could include information like 'cooked this month', 'what was eaten most' and other insights. We did not manage to implement this, but the data is already available. Only the web interface and a dashboard module would have to be integrated.

Possible extensions that could be implemented in the future are:

- A 'first use setting' feature: When first launching the app, a user can add every item they already have in their fridge so that the recipes suggested are more accurate.
- Make the application available for installation
- Create application for smartphones or other mobile devices, that allows the user to directly take a picture of their grocery receipt for scanning

7. Conclusion

Through the full development of the MySmartFridge project, we gained valuable practical experience at both the technical and methodological levels. From a technical perspective, the project strengthened our understanding of core Python concepts such as data structures, object-oriented programming, and modular system design. By centering the application around the Fridge class, features such as inventory management, OCR-based receipt scanning, and recipe generation could be developed around a shared data structure, which contributed to the overall scalability and consistency of the system. In addition, when dealing with noisy OCR outputs and the inherent unpredictability of Ollama-generated results, we became aware of the importance of robust system design. As a result, we introduced rule-based filtering, regular expression parsing, and comprehensive exception handling across multiple modules to ensure that the application remains functional even in failure scenarios.

From a methodological standpoint, the project taught us how to make informed trade-offs between deterministic algorithms and intelligent models. We prioritized rule-based text parsing and data cleaning to address the majority of structured problems, and only introduced a local LLM as a controlled post-processing tool when semantic issues could not be reliably handled by rules alone. This layered and incremental design approach reduced unnecessary reliance on the model while improving the interpretability and maintainability of the system. Furthermore, the iterative development process from V0 to V2 highlighted the importance of first building a minimal working system and then progressively extending functionality and improving the user interface. Overall, this project allowed us to apply

Python techniques learned in class to a complete real-world application, while also strengthening our teamwork and engineering design skills.